# ASSIGNMENT-15.1

Name: B.charan

HT. No: 2303A51623

Batch: 8

## Task 1: Student Records API

Scenario:

In this task, AI-assisted coding (GitHub Copilot) was used to generate a RESTful API for managing student records using Flask. The AI was prompted to create CRUD endpoints with in-memory storage and JSON responses.

Prompt:

# Build a RESTful API in Flask to manage student records

# Endpoints: GET /students, POST /students, PUT /students/{id}, DELETE /students/{id} # Use

an in-memory dic onary to store records. Return JSON responses.

Code:

```
#Task 1: Student Records API  from flask import Flask,
jsonify, request app = Flask(__name__) students = {}
next_id = 1

@app.route('/students', methods=['GET']) def get_students():
return jsonify(list(students.values()))

@app.route('/students', methods=['POST']) def
add_student():    global next_id
   data = request.get_json()    students[next_id] = {'id': next_id,
**data}    next_id += 1    return jsonify(students[next_id - 1]), 201

@app.route('/students/<int:sid>', methods=['PUT']) def update_student(sid):
if sid not in students:        return jsonify({'error': 'Not found'}), 404
students[sid].update(request.get_json())    return jsonify(students[sid])

@app.route('/students/<int:sid>',

methods=['DELETE']) def delete_student(sid):    if sid not in students:
return  jsonify({'error': 'Not found'}), 404    return
jsonify(students.pop(sid)) if __name__

== '__main__':    app.run(debug=True)
```

Result:

```
* Running on http://127.0.0.1:5000
```

```
GET /students        -> 200 []
POST /students       -> 201 {"id":1,"name":"Alice","roll":"23CS001"}
GET /students        -> 200 [{"id":1,"name":"Alice","roll":"23CS001"}]
PUT /students/1      -> 200 {"id":1,"name":"Alice","roll":"23CS001","grade":"A"}
DELETE /students/1   -> 200 {"id":1,"name":"Alice","roll":"23CS001","grade":"A"}
```

Observa on:

The AI-generated Flask API successfully implemented all four CRUD opera ons for student records. The inmemory dic onary allowed fast reads and writes. The AI correctly inferred the need for a global ID counter and structured the response in JSON format. This shows that AI can reliably scaffold REST APIs when given clear endpoint specifica ons.

## Task 2: Library Book Management API

Scenario:

A RESTful API for managing library books was generated using AI. The task required CRUD opera ons plus par al updates (PATCH) and error handling for invalid requests.

Prompt:

# Write a Flask REST API for a library book management system

# Example endpoints: GET /books, POST /books, GET /books/1, PATCH /books/1, DELETE /books/1 # PATCH updates par al fields like availability. Handle 404 errors.

Code:

```
#Task 2: Library Book Management API from flask
import Flask, jsonify, request app =
Flask(__name__) books = {} next_id =
1

@app.route('/books', methods=['GET']) def get_books():
return jsonify(list(books.values()))

@app.route('/books', methods=['POST']) def add_book():     global next_id       data
= request.get_json()     books[next_id] = {'id':
next_id, 'available': True, **data}     next_id += 1     return jsonify(books[next_id - 1]), 201

@app.route('/books/<int:bid>', methods=['GET']) def get_book(bid):     if bid not in books:
return jsonify({'error': 'Book not found'}), 404     return jsonify(books[bid])

@app.route('/books/<int:bid>', methods=['PATCH']) def update_book(bid):
```

```
    if bid not in books:        return jsonify({'error': 'Book not found'}), 404
books[bid].update(request.get_json())     return jsonify(books[bid])

@app.route('/books/<int:bid>', methods=['DELETE']) def
delete_book(bid):     if bid not in books:        return jsonify({'error': 'Book
not found'}), 404     return jsonify(books.pop(bid)) if __name__ ==
'__main__':      app.run(debug=True)
```

Result:

```
 * Running on http://127.0.0.1:5000
POST /books         -> 201 {"id":1,"title":"Python 101","available":true}
GET /books/1        -> 200 {"id":1,"title":"Python 101","available":true}
PATCH /books/1      -> 200 {"id":1,"title":"Python 101","available":false} GET /books/99        -> 404 {"error":"Book not found"}
DELETE /books/1     -> 200 {"id":1,"title":"Python 101","available":false}
```

Observa on:

The AI-generated PATCH endpoint correctly handled par al updates, only modifying the fields provided in the request body. Error handling for non-existent records returned proper 404 responses. Compared to Task 1, the PATCH method offers finer control over updates, demonstra ng that AI can generate produc on-style pa erns when given examples in the prompt.

## Task 3: Employee Payroll API

Scenario:

AI was used to design an employee payroll management API. The AI was addi onally asked to suggest a data model and add docstrings/comments for all endpoints, demonstra ng AI's capability for documenta on genera on.

Prompt:

# Create a Flask API for employee payroll management

# Endpoints: GET /employees, POST /employees, PUT /employees/{id}/salary, DELETE /employees/{id} # AI: suggest data model structure and add docstrings for each endpoint

Code:

```
#Task 3: Employee Payroll API from flask import Flask, jsonify,
request

app = Flask(__name__)
# Data model: {id, name, department, salary} employees = {}
next_id = 1

@app.route('/employees', methods=['GET']) def get_employees():
    """Return list of all employees."""
```

```
    return jsonify(list(employees.values())) @app.route('/employees',
methods=['POST']) def add_employee():
    """Add new employee with salary details."""      global next_id
data =
request.get_json()     employees[next_id] = {
'id': next_id,
        'name': data.get('name', ''),
        'department': data.get('department', ''),
        'salary': data.get('salary', 0)       }      next_id += 1      return
jsonify(employees[next_id - 1]), 201
@app.route('/employees/<int:eid>/salary',   methods=['PUT']) def
update_salary(eid):     """Update salary of a specific employee."""     if eid not
in employees:         return jsonify({'error':
'Not found'}), 404
    employees[eid]['salary'] = request.get_json().get('salary')     return jsonify(employees[eid])
@app.route('/employees/<int:eid>', methods=['DELETE']) def
delete_employee(eid):     """Remove an employee from the system."""     if eid
not in employees:         return jsonify({'error':
'Not found'}), 404     return
jsonify(employees.pop(eid)) if __name__
== '__main__':
app.run(debug=True)
```

Result:

```
 * Running on http://127.0.0.1:5000
POST /employees      -> 201 {"id":1,"name":"Ravi","department":"IT","salary":50000}
GET /employees       -> 200 [{"id":1,"name":"Ravi","department":"IT","salary":50000}]
PUT /employees/1/salary -> 200 {"id":1,"name":"Ravi","department":"IT","salary":65000}  DELETE /employees/1  -> 200
{"id":1,"name":"Ravi","department":"IT","salary":65000}
```

Observa on:

The AI not only generated the API endpoints but also suggested a clean data model with four fields (id, name, department, salary) and automa cally added docstrings to each func on. The dedicated PUT /salary endpoint allows salary updates without overwri ng other employee data. This demonstrates that AI-assisted coding improves both code quality and documenta on simultaneously.

## Task 4: Real-Time Applica on – Online Food Ordering API

Scenario:

In this task, AI was used to design a full food ordering system API. The AI was also prompted to suggest improvements such as authen ca on and pagina on, simula ng a real-world scenario.

Prompt:

# Design a Flask REST API for an online food ordering system

# Endpoints: GET /menu, POST /order, GET /order/{id}, PUT /order/{id}, DELETE /order/{id} # AI: generate REST code and suggest improvements like authen ca on and pagina on

Code:

```
#Task 4: Online Food Ordering API from flask import Flask,

jsonify, request app

= Flask(__name__) menu = [

    {'id': 1, 'name': 'Biryani', 'price': 180},
{'id': 2, 'name': 'Dosa',    'price': 60},       {'id': 3, 'name': 'Burger',  'price':
120},
] orders = {}
next_order_id =
1

@app.route('/menu', methods=['GET']) def get_menu():
    """List all available dishes."""     return jsonify(menu)

@app.route('/order', methods=['POST']) def
place_order():       """Place a new food order."""
global next_order_id     data = request.get_json()
orders[next_order_id] = {
        'order_id': next_order_id,
        'items': data.get('items', []),
        'status': 'Placed'
    }     next_order_id += 1     return jsonify(orders[next_order_id - 1]), 201

@app.route('/order/<int:oid>', methods=['GET']) def track_order(oid):     """Track status
of an order."""     if oid not in orders:         return jsonify({'error': 'Order not found'}), 404
return jsonify(orders[oid])

@app.route('/order/<int:oid>', methods=['PUT']) def update_order(oid):     """Update an
existing order."""     if oid not in orders:         return jsonify({'error':
'Order not found'}), 404
    orders[oid].update(request.get_json())     return jsonify(orders[oid])

@app.route('/order/<int:oid>', methods=['DELETE']) def cancel_order(oid):
"""Cancel an order."""     if oid not in orders:         return jsonify({'error': 'Order not
found'}), 404     return jsonify(orders.pop(oid))

# AI Suggested Improvements:
# 1. Add JWT authentication to protect POST/PUT/DELETE endpoints
# 2. Add pagination to GET /menu (e.g., ?page=1&limit=10)
# 3. Add order status transitions (Placed -> Preparing -> Delivered) #
4. Connect to a persistent database (PostgreSQL/MongoDB)
```

```
if __name__ == '__main__':
app.run(debug=True)
```

Result:

```
* Running on http://127.0.0.1:5000
```

```
GET /menu          -> 200 [{"id":1,"name":"Biryani","price":180}, ...]
POST /order        -> 201 {"order_id":1,"items":[1,2],"status":"Placed"}
GET /order/1       -> 200 {"order_id":1,"items":[1,2],"status":"Placed"}
PUT /order/1       -> 200 {"order_id":1,"items":[1,3],"status":"Placed"}
DELETE /order/1    -> 200 {"order_id":1,"items":[1,3],"status":"Placed"}
GET /order/99      -> 404 {"error":"Order not found"}
```

Observa on:

The AI generated a fully working food ordering API including a pre-seeded menu and order management with status tracking. Addi onally, the AI proac vely suggested real-world improvements: JWT authen ca on to secure endpoints, pagina on for large menus, order status transi ons, and database integra on. This demonstrates that AI-assisted coding goes beyond basic code genera on by providing architectural recommenda ons suited for produc on environments.